SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA0303
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr DUMPALA PRASANTH

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Simulation-Based Assessment Question

Duration: 1hr

1. Simulate a **Hierarchical Reinforcement Learning (HRL)** agent for autonomous warehouse navigation.

   (5 Marks)

2. Implement a **Hierarchical Abstract Machine (HAM)** to control a robot performing sequential tasks.

   (5 Marks)

3. Develop the **MAXQ Value Decomposition** algorithm for a taxi navigation environment.

   (5 Marks)

4. Simulate a **Meta-Reinforcement Learning** agent that rapidly adapts to multiple GridWorld environments.

   (5 Marks)

5. Design a **Multi-Agent Reinforcement Learning (MARL)** system for cooperative robot path planning.

   (5 Marks)

6. Implement a **competitive Multi-Agent RL** environment for autonomous drone navigation.

   (5 Marks)

7. Simulate a **Partially Observable Markov Decision Process (POMDP)** for autonomous vehicle navigation under limited visibility.

(5 Marks)

8. Design an **ethical Reinforcement Learning** simulation for autonomous decision-making while satisfying safety constraints.

(5 Marks)

9. Develop an RL-based **adaptive traffic signal control** system and evaluate its performance.

10. Simulate an RL-based **smart energy management system** for optimizing power distribution in a microgrid.. **.**

(5 Marks)

Code of Conduct:

I ___***K Uday Kiran Reddy (192425211)***___ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:kuday

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively

1. Simulate a Hierarchical Reinforcement Learning (HRL) Agent for Autonomous Warehouse Navigation

Aim

To simulate an HRL agent that navigates a warehouse by dividing a complex navigation task into smaller subtasks.

Concept

Hierarchical Reinforcement Learning (HRL) decomposes a large problem into multiple levels. A high-level policy selects a subtask, while a low-level policy performs the required primitive actions.

For example:

High-level tasks:

- Go to storage area.
- Pick package.
- Go to delivery area.
- Deliver package.

Low-level actions:

- Move up.
- Move down.
- Move left.
- Move right.
- Stop.

State

The state can contain:

- Robot position.
- Package location.
- Destination.
- Obstacles.
- Battery level.

Action

The high-level agent selects a subtask, while the low-level controller selects movement actions.

Reward

$$R = R_{delivery} - R_{collision} - R_{distance}$$

Example:

- Successful delivery = +100
- Collision = -50
- Each unnecessary movement = -1

Simulation Workflow

Warehouse Environment

↓

High-Level Policy

↓

Select Subtask

↓

Low-Level Policy

↓

Select Movement

↓

Move Robot Safely

↓

Receive Reward

↓

Update Policies

Expected Result

The HRL agent learns to complete warehouse tasks efficiently by first selecting the correct subtask and then performing the required navigation actions.

Conclusion

HRL reduces the complexity of warehouse navigation by separating **task planning** from **low-level movement control**, making it suitable for complex robotic environments.

2. Implement a Hierarchical Abstract Machine (HAM) to Control a Robot Performing Sequential Tasks

Aim

To implement a Hierarchical Abstract Machine for controlling a robot through a sequence of predefined tasks.

Concept

A **Hierarchical Abstract Machine (HAM)** combines reinforcement learning with a structured hierarchy of tasks. Instead of allowing the agent to perform any action at any time, a machine specifies which actions are valid in each task.

Example Task Sequence

Start

↓

Navigate to Package

↓

Pick Package

↓

Navigate to Destination

↓

Drop Package

↓

Finish

HAM Components

1. **State:** Robot location, package status and destination.
2. **Machine:** Defines task hierarchy.
3. **Action:** Movement or task execution.
4. **Reward:** Reward for successful task completion.
5. **Policy:** Selects actions within permitted machine states.

Example

ROOT

├── Navigate

│ ├── Move Left

│ ├── Move Right

│ └── Move Forward

│

├── Pickup
│ └── Pick Package
│
├── Deliver
│ └── Move to Destination
│
└── Drop
└── Release Package

Reward

$$R = \begin{cases} +100 & \text{Successful delivery} \\ -20 & \text{Wrong task} \\ -1 & \text{Each movement} \end{cases}$$

Expected Result

The robot follows the hierarchical task structure and completes the required tasks in the correct order.

Conclusion

HAM is useful when a problem has a known task structure because it restricts inappropriate actions while allowing RL to learn the best decisions within the hierarchy.

3. Develop the MAXQ Value Decomposition Algorithm for a Taxi Navigation Environment

Aim

To implement MAXQ value decomposition for solving a hierarchical taxi navigation problem.

Concept

MAXQ decomposes a complex reinforcement learning problem into smaller subtasks. Each subtask has its own value function.

For a taxi environment, the task can be divided into:

1. Navigate to passenger.
2. Pick up passenger.
3. Navigate to destination.
4. Drop passenger.

Hierarchy

Taxi Delivery

|

+---- Go to Passenger

|

+---- Pickup

|

+---- Go to Destination

|

+---- Drop

State

The state includes:

- Taxi position.
- Passenger position.
- Destination position.
- Passenger pickup status.

Actions

- Move north.
- Move south.
- Move east.
- Move west.
- Pickup.
- Drop.

MAXQ Value

The overall value is decomposed into values of subtasks:

$$V(s) = V_1(s) + V_2(s) + \ldots + V_n(s)$$

The agent learns values for individual subtasks rather than learning one large value function.

Reward

Example:

- Successful passenger delivery = +100
- Illegal pickup/drop = -10
- Each movement = -1

Expected Result

The taxi agent learns a sequence of subtasks that enables it to pick up and deliver passengers efficiently.

Conclusion

MAXQ reduces the complexity of learning by decomposing the taxi problem into reusable subtasks, improving learning efficiency and interpretability.

4. Simulate a Meta-Reinforcement Learning Agent That Rapidly Adapts to Multiple GridWorld Environments

Aim

To simulate a Meta-RL agent that can quickly adapt to different GridWorld environments.

Concept

Meta-Reinforcement Learning trains an agent to learn how to learn. Instead of training from scratch for every environment, the agent learns a general strategy that allows rapid adaptation.

Example Environments

Environment A → Goal at Top-Right

Environment B → Goal at Bottom-Left

Environment C → Different Obstacles

Environment D → Different Rewards

State

- Agent position.
- Goal position.
- Obstacles.
- Previous observations.
- Previous rewards.

Actions

- Up.
- Down.
- Left.
- Right.

Reward

- Reaching goal = +100
- Hitting obstacle = -10
- Each movement = -1

Meta-Learning Process

Multiple GridWorld Tasks

↓

Meta Training

↓

Learn General Policy

↓

New GridWorld

↓

Few Adaptation Steps

↓

Improved Performance

Expected Result

The agent should require fewer training steps when presented with a new but related GridWorld environment.

Conclusion

Meta-RL improves adaptability because the agent learns general strategies that can be transferred to new environments.

5. Design a Multi-Agent Reinforcement Learning System for Cooperative Robot Path Planning

Aim

To design a MARL system where multiple robots cooperate to reach their destinations without collisions.

Concept

Multi-Agent Reinforcement Learning (MARL) involves multiple learning agents interacting within the same environment.

Agents

For example:

- Robot 1
- Robot 2
- Robot 3

Each robot has its own position and destination.

State

Each robot observes:

- Own position.
- Destination.
- Positions of nearby robots.

- Obstacles.
- Available paths.

Actions

- Move up.
- Move down.
- Move left.
- Move right.
- Stay.

Reward

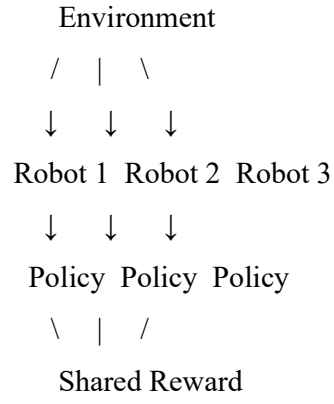
A cooperative reward can be:

$$R = R_{goal} - R_{collision} - R_{distance}$$

Example:

- Reaching destination = +50
- Collision = -50
- Short movement = small positive/low penalty.
- Successful group completion = +100.

Architecture



Expected Result

The robots learn to coordinate their routes, avoid collisions and reach their destinations efficiently.

Conclusion

MARL is suitable for cooperative robot navigation because multiple agents can learn coordinated behavior rather than planning independently.

6. Implement a Competitive Multi-Agent RL Environment for Autonomous Drone Navigation

Aim

To simulate multiple autonomous drones that compete for routes or delivery objectives while avoiding collisions.

Concept

In **competitive MARL**, agents have objectives that may conflict with each other. Each drone tries to maximize its own reward.

State

Each drone observes:

- Current position.
- Destination.
- Battery level.
- Other drone locations.
- Obstacles.
- Restricted zones.

Actions

- Move forward.
- Turn left.
- Turn right.
- Increase altitude.
- Decrease altitude.
- Hover.

Reward

For each drone:

$$R_i = R_{delivery} - R_{collision} - R_{delay}$$

Example:

- Successful delivery = +100
- Collision = -100
- Entering restricted zone = -100
- Excessive delay = -1 per time step.

Simulation

Drone 1 ⟶ Destination 1

↘

Shared Airspace

↗

Drone 2 ⟶ Destination 2

Each drone learns its policy while considering the actions of other drones.

Expected Result

The drones learn routes that maximize delivery performance while avoiding collisions and restricted areas.

Conclusion

Competitive MARL is useful when autonomous agents operate in shared environments and must make decisions while considering the behavior of other agents.

7. Simulate a POMDP for Autonomous Vehicle Navigation Under Limited Visibility

Aim

To simulate autonomous vehicle navigation when the vehicle cannot completely observe its environment.

Concept

A **Partially Observable Markov Decision Process (POMDP)** is used when the agent cannot directly observe the complete state of the environment.

For example, fog may prevent an autonomous vehicle from seeing distant objects.

POMDP Components

Component	**Example**
States	Vehicle position, pedestrians, other vehicles
Observations	Camera/radar observations
Actions	Accelerate, brake, steer
Rewards	Safe and efficient driving
Transition	Vehicle movement
Observation probability	Sensor uncertainty

Belief State

Instead of knowing the exact state, the vehicle maintains a probability distribution:

$$b(s) = P(s \mid observation)$$

For example:

Possible pedestrian location:

Left side → 20%

Center → 60%

Right side → 20%

Reward

- Safe driving = positive reward.
- Reaching destination = +100.
- Collision = -1000.
- Sudden unsafe action = -100.

Simulation Workflow

Environment

↓

Partial Observation

↓

Belief State

↓

RL Policy

↓

Driving Action

↓

New Observation

Expected Result

The vehicle makes safer decisions despite incomplete environmental information.

Conclusion

POMDP-based RL is suitable for autonomous vehicles operating under fog, darkness, sensor noise or other conditions where the complete state cannot be directly observed.

8. Design an Ethical Reinforcement Learning Simulation for Autonomous Decision-Making While Satisfying Safety Constraints

Aim

To design an RL system that makes decisions while satisfying ethical and safety requirements.

Concept

An ethical RL system should not optimize rewards without considering safety, fairness and human well-being.

Example

Consider an autonomous robot that must choose between different routes.

State

- Robot position.
- Obstacles.
- People nearby.
- Battery.
- Environmental risk.

Actions

- Move left.
- Move right.
- Stop.
- Take an alternative route.

Reward

$$R = R_{task} - P_{risk} - P_{harm}$$

where:

- R_{task}= task completion reward.
- P_{risk}= safety penalty.
- P_{harm}= penalty for potentially harmful actions.

Safety Constraint

The system should satisfy:

$$Risk(s, a) \leq Risk_{max}$$

Unsafe actions should be blocked even if they provide a higher immediate reward.

Ethical Principles

1. Human safety.
2. Fair decision-making.

3. Transparency.
4. Privacy.
5. Accountability.
6. Human override.

Architecture

Environment

↓

RL Policy

↓

Safety/Ethics Layer

↓

Safe Action

↓

Environment

↓

Reward

Expected Result

The agent completes tasks while avoiding actions that violate predefined safety and ethical constraints.

Conclusion

Ethical RL should combine reward optimization with **hard safety constraints and human oversight** to prevent harmful behavior.

9. Develop an RL-Based Adaptive Traffic Signal Control System and Evaluate Its Performance

Aim

To develop an RL agent that dynamically controls traffic lights to reduce congestion.

Concept

Traditional traffic lights use fixed timings. RL can dynamically change signal durations according to current traffic conditions.

State

The state may include:

- Number of vehicles.
- Queue length.
- Waiting time.
- Current signal.
- Traffic flow rate.
- Pedestrian status.

Actions

The agent can:

- Keep the current green signal.
- Change the signal.
- Extend green time.
- Reduce green time.

Reward

$$R = -(\text{Queue Length} + \text{Waiting Time})$$

The system receives higher rewards when congestion and waiting time decrease.

Workflow

Traffic Sensors

↓

Observe Traffic State

↓

RL Agent

↓

Select Signal Action

↓

Traffic Lights

↓

Measure Waiting Time

↓

Reward

↓

Update Policy

Performance Evaluation

The system can be evaluated using:

Metric	**Expected Improvement**
Average waiting time	Decrease
Queue length	Decrease
Vehicle throughput	Increase
Number of stops	Decrease
Fuel consumption	Decrease

Example

Before RL:

Average waiting time = 60 seconds

After RL:

Average waiting time = 35 seconds

This indicates improved traffic flow.

Conclusion

RL-based traffic signal control can adapt to changing traffic patterns and reduce congestion more effectively than fixed-time signal control.

10. Simulate an RL-Based Smart Energy Management System for Optimizing Power Distribution in a Microgrid

Aim

To simulate an RL system that manages electricity distribution in a microgrid while minimizing energy costs and maintaining supply reliability.

Concept

A microgrid may contain:

- Solar panels.
- Wind turbines.
- Battery storage.
- Grid connection.
- Household/industrial loads.

RL can learn when to use, store or purchase energy.

State

The state can include:

- Current electricity demand.
- Solar generation.
- Wind generation.
- Battery charge level.
- Electricity price.
- Grid availability.
- Time of day.

Actions

The RL agent can:

1. Use renewable energy.
2. Charge battery.
3. Discharge battery.
4. Purchase power from the grid.
5. Supply power to loads.

Reward Function

A suitable reward is:

$$R = -C_{energy} + R_{renewable} - P_{shortage}$$

where:

- C_{energy}= electricity cost.
- $R_{renewable}$= reward for renewable energy utilization.
- $P_{shortage}$= penalty for power shortage.

Example

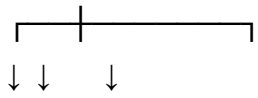
Solar/Wind

↓

Microgrid Controller

↓

RL Agent



Load Battery Grid

Battery Constraint

The system must ensure:

$$SOC_{min} \leq SOC_t \leq SOC_{max}$$

where SOCrepresents the battery state of charge.

Performance Evaluation

The system can be evaluated using:

- Total energy cost.
- Renewable energy utilization.
- Battery efficiency.
- Power shortage.
- Grid dependency.
- Peak demand.

Expected Result

The RL agent learns to use renewable energy when available, store excess energy in batteries, and purchase electricity from the grid when necessary.

Conclusion

RL can provide adaptive energy management for microgrids by balancing **energy cost, renewable utilization, battery storage and power reliability**.